

Monetization Orchestrator: Async Ads, IAP & LiveOps

Complete Setup, SDK transition, Console Configurations, and Analytics Tracking.

This guide provides a comprehensive walkthrough for configuring and extending the **MonetizationOrchestrator** in your projects.

📋 Requirements

- **Minimum Unity Version:** 2021.3 LTS
- **Supported Platforms:** iOS, Android
- **Dependencies (Optional):** UniTask (com.cysharp.unitask), Unity IAP (com.unity.purchasing)

📖 Table of Contents

1. [Quick Start Setup Guide](#)
2. [Introduction to Mock Editor Modes](#)
3. [LiveOps: Google Sheets Remote Config](#)
4. [Migrating to Battle-Ready SDKs](#)
5. [No-Code Setup \(Unity Events Inspector\)](#)
6. [AAA Architecture \(Dependency Injection & UniTask\)](#)
7. [App Store Connect & Google Play Console Configurations](#)
8. [Custom Analytics & Trackers \(Impression Tracking\)](#)
9. [Script Reference](#)
10. [Frequently Asked Questions \(FAQ\)](#)

1. Quick Start Setup Guide

Follow this step-by-step tutorial to get ads running in your game within 5 minutes.

Step 1: Import the Package Drag the `.unitypackage` into your Unity project or import it via the Package Manager.

Step 2: Setup Resources Go to the top menu bar and select `Tools -> Monetization Orchestrator -> Setup Resources & Prefabs`. This will generate the necessary configuration files.

Step 3: Add to Scene Navigate to the `Assets/Plugins/MonetizationOrchestrator/Prefabs` folder. Drag the `MonetizationOrchestrator` prefab into your first boot scene (e.g., your Main Menu or Loading screen).

Step 4: Test in Editor Drag the `MonetizationDemoCanvas` prefab from the `Prefabs` folder into your scene. Hit **Play** in the Unity Editor, and click "Show Rewarded Ad". The Mock System works immediately!

2. Introduction to Mock Editor Modes

To speed up development cycles and reduce overhead, **MonetizationOrchestrator** is equipped with a complete, SDK-free editor simulation system.

How it works:

- **Mock Ads:** When you invoke `AdManager.Instance.ShowRewardedAdAsync(...)` in the Editor, the system automatically checks for SDK symbols. Finding none, it spawns `MockAdCanvas` from Resources. The Canvas triggers a dark screen blocker, pauses `Time.timeScale` to 0, mutes `AudioListener.pause = true`, and runs a visual progress slider.
 - Click **Claim Reward**: Simulates ad success, firing `OnRewardedAdCompleted`.
 - Click **Skip/Close**: Simulates early exit, firing `OnRewardedAdFailed`.
- **Visual Mock Banners:** Calling `AdManager.Instance.ShowBanner()` in Editor mode dynamically generates a highly-visible, screen-space Canvas overlay at the bottom (or top) of the screen. This allows you to test your UI Safe Areas without needing real ads!
- **Mock IAP:** When you call `IAPManager.Instance.BuyProduct("id")`, it bypasses native billing stores and runs a simulated 1.5s timer. It then fires `OnPurchaseCompleted` with a randomized mock transaction ID.

3. LiveOps: Google Sheets Remote Config

Balance your ad intervals and cooldowns without forcing players to download an update! This package includes a lightweight `RemoteConfigManager` that parses live Google Sheets.

Step-by-Step Setup:

1. Create a new Google Sheet.
2. In cell **A1** type `Key` , and in cell **B1** type `Value` .
3. Add configuration pairs. For example:
 - `interstitial_interval` (Column A) : `90` (Column B)
 - `rewarded_cooldown` (Column A) : `30` (Column B)
4. Go to **File -> Share -> Publish to web**.
5. Select the specific Sheet and choose **Comma-separated values (.csv)** as the format. Click Publish.
6. Copy the provided link.
7. Select the `MonetizationOrchestrator` prefab in your scene and paste the link into the `Csv Url` field of the `RemoteConfigManager` component.

Every time the game launches, it will download these values and seamlessly override the ad timers. *Note: A live Demo URL is already pre-configured out-of-the-box so you can test it immediately!*

Supported Keys out-of-the-box:

- `interstitial_min_interval` (float)
- `rewarded_cooldown_seconds` (float)
- `initial_ad_delay` (float)
- `interstitial_game_over_freq` (int)
- `interstitial_level_complete_freq` (int)

(Note: You can add any other custom keys to your table like `rewarded_coins` or `onboarding_ad_level` and read them safely in your own game scripts. The API provides foolproof fallback values to prevent crashes if the player is offline or the key is missing from the sheet.)

```
// Safe parsing with default fallback values (Crash-free):
int coins = RemoteConfigManager.Instance.GetIntConfig("rewarded_coins", defaultValue: 60).Value;
bool isTimerBased = RemoteConfigManager.Instance.GetBoolConfig("interstitial_timer_based", defaultValue: true).Value;
float delay = RemoteConfigManager.Instance.GetFloatConfig("initial_ad_delay", defaultValue: 5.5f).Value;
```

REPLACE THE DEMO URL BEFORE PUBLISHING: The pre-configured Google Sheet URL is for demonstration purposes only. If you do not replace it with your own Google Sheet link, your game's ad pacing in production will be controlled by our public demo table!

4. Migrating to Battle-Ready SDKs

When you are ready to publish your game, you can easily switch from Mock modes to native SDK platforms.

Step 1: Install the Packages

Our orchestrator architecture relies on the `IAdProvider` pattern. It automatically detects and binds to whichever major SDK you install into your project:

- **For AppLovin MAX:** Import the MAX Unity Plugin. (Industry favorite for midcore/casual).
- **For IronSource LevelPlay:** Import the LevelPlay Mediation package using Unity Package Manager (UPM).
- **For Unity Ads:** Import the Unity Ads Mediation package via UPM.
- **For Google AdMob:** Import the AdMob Unity package.

Step 2: Apply Integrations

For **Unity IAP** and **UniTask**, simply go to `Tools -> Monetization Orchestrator -> SDK Integrations` and click **"Apply Integrations"**. The custom Editor Window will automatically configure the assembly definitions for you safely!

For Ad SDKs, go to `Edit -> Project Settings -> Player -> Scripting Define Symbols` and add the respective define for your ad network:

1. `MONETIZATION_APPLOVIN`
2. `MONETIZATION_LEVELPLAY`
3. `MONETIZATION_UNITYADS`
4. `MONETIZATION_ADMOB`

Step 3: Configure AdManager

Select the `MonetizationOrchestrator` prefab in your boot scene. In the inspector:

- **Android & iOS Ad Unit IDs:** Enter your platform-specific placement IDs (Rewarded, Interstitial, Banner). The plugin will automatically detect the device OS and use the correct ID at runtime!
- **Enable Banners:** Uncheck this to completely disable banner logic and loading if your game doesn't use banners.
- **Enable Test Suite:** Extremely important! Leave this ON during development so `OpenTestSuite()` can launch the Mediation Debugger on your physical device. **TURN THIS OFF** before publishing to production!

5. No-Code Setup (Unity Events Inspector)

You don't need to write a single line of C# code to connect ads and purchases to your game economy!

MonetizationOrchestrator comes with a dedicated `MonetizationUnityEvents` component that exposes all reward callbacks directly to the Unity Inspector.

Setup Instructions:

1. Create a new `GameObject` in your UI scene, or select an existing one (like your `ShopCanvas`).
2. Click **Add Component** and search for `Monetization Unity Events`.
3. You will now see Inspector slots for `On Rewarded Ad Completed`, `On Interstitial Ad Shown`, and `On Purchase Completed`.
4. Click the **+** icon on `On Rewarded Ad Completed`.
5. Drag your `GameManager` or `EconomyManager` script into the object slot.
6. Select your reward method from the dropdown (e.g., `GameManager.AddCoins(int)`).

The plugin will automatically pass the correct reward amount (configured in your Ad Network / Remote Config) directly to your method!

6. AAA Architecture (Dependency Injection & UniTask)

This plugin is designed with large-scale project standards in mind (AAA architecture).

Dependency Injection Support (VContainer / Zenject)

The `AdManager` and `IAPManager` implement the `IAdManagerService` and `IIAPManagerService` interfaces. Instead of calling `AdManager.Instance`, you can cleanly inject them into your Dependency Injection container.

Example for **VContainer**:

```
public class MonetizationLifetimeScope : LifetimeScope
{
    protected override void Configure(IContainerBuilder builder)
    {
        builder.RegisterComponent(AdManager.Instance).As<IAdManagerService>();
        builder.RegisterComponent(IAPManager.Instance).As<IIAPManagerService>();
    }
}
```

UniTask Support

If you have the **UniTask** package installed in your project, the plugin will automatically detect it and add the `MONETIZATION_UNITASK` scripting define. Once active, all plugin methods (e.g., `ShowRewardedAdAsync`) will return a zero-allocation `UniTask<T>` instead of the standard C# `Task<T>`. This guarantees **zero memory allocations** during ad displays, which is critical for preventing micro-stutters (GC Spikes) on mobile devices.

7. App Store Connect & Google Play Console Configurations

To receive active billing transactions, your product IDs configured in the `ShopOfferSO` ScriptableObjects must exactly match the product IDs defined in the developer console portals.

Google Play Console Setup:

1. Select your application, navigate to **Monetize -> In-app products**.
2. Click **Create product**:
 - **Product ID**: Enter a unique reverse-dns ID (e.g., `com.company.game.doublecoins`).
 - **Name & Description**: Localized text displayed in the shop.
 - **Default Price**: Set your pricing tier.
3. Click **Save** and then **Activate**.
4. In `ShopOfferSO`, assign `iapProductId` to matches this exactly: `com.company.game.doublecoins`.

App Store Connect Setup:

1. Open your app dashboard, go to **In-App Purchases -> Manage**.
2. Click the **+** button:
 - Select product type (Consumable, Non-Consumable).
 - **Product ID**: Enter the matching ID (e.g. `com.company.game.doublecoins`).
 - **Pricing**: Select the App Store pricing tier.
3. Fill in the localization details, upload a screenshot for review, and click **Save**.

8. Custom Analytics & Trackers (Impression Tracking)

Every serious publisher needs to track **Impression-Level Revenue Data (ILRD)** to calculate exact LTV and ROI on acquisition campaigns.

MonetizationOrchestrator is completely decoupled from any specific analytics framework (like Firebase SDK, Tenjin, GameAnalytics, etc.). Instead, it exposes a clean

C# event callback:

```
public static event Action<AdImpressionData> OnImpressionDataTracked;
```

Simply create a helper component in your project and subscribe to the event. The `AdImpressionData` object contains the `AdNetwork`, `AdUnit`, `Revenue`, and `Currency` provided by the mediation SDK.

9. Script Reference

If you prefer to write code rather than using the Unity Events Inspector, you can interface directly with the core services.

`AdManager` (OR `IAdManagerService`)

The primary service for displaying advertisements.

- `UniTask<bool> ShowRewardedAdAsync(string placementId)` : Attempts to show a rewarded ad. Returns `true` if the user fully watched the ad, and `false` if they skipped it or it failed to load.
- `UniTask<bool> ShowInterstitialAsync(string placementId)` : Attempts to show an interstitial ad. Subject to the `interstitial_interval` cooldown. Returns `true` if successfully displayed.
- `void ShowBanner()` : Displays a banner ad.
- `void HideBanner()` : Hides the current banner ad.

`IAPManager` (OR `IIAPManagerService`)

The primary service for handling In-App Purchases.

- `UniTask<bool> BuyProduct(string productId)` : Initiates a purchase flow for the specified product ID. Returns `true` if the purchase was successful and verified.
- `string GetProductPriceString(string productId)` : Returns the localized price string (e.g., "\$1.99" or "€1.99") for the given product ID to display on your UI buttons.

`AdTimerService`

A utility service that handles pacing.

- `bool IsInterstitialReady()` : Checks if enough time has passed since the last interstitial ad.
- `float GetTimeUntilNextInterstitial()` : Returns the remaining cooldown in seconds.

10. Frequently Asked Questions (FAQ)

Q: I installed UniTask / Unity IAP, but the integration isn't working or throws assembly errors.

A: Make sure you open `Tools -> Monetization Orchestrator -> SDK Integrations` and click **"Apply Integrations"**. The plugin relies on this tool to safely configure the assembly definitions without causing hard dependency errors.

Q: How do I test purchases on Android without being charged real money?

A: Ensure `Mock Mode` is checked on `IAPManager` for editor simulation. For on-device testing, add your Google account to the **License Testing** section in your Google Play Console under Setup. You can then select "Test Card, always approves" during billing prompts.

Q: My banner doesn't display or is cut off.

A: The system automatically retrieves banner heights (standard 50dp). Ensure your Canvas UI has a safe area layout fitter at the bottom (or top if using `BannerAtTop = true`) to prevent elements from overlapping with active banner ads.

📧 Support & Feedback

If you have any questions, bug reports, or feature requests, feel free to reach out to our support channel.

📧 **Enjoying Monetization Orchestrator?** Please consider leaving a review on the Asset Store!